

```
#!/usr/bin/env python3
"""Improve a skill description based on eval results.

Takes eval results (from run_eval.py) and generates an improved description
by calling `claude -p` as a subprocess (same auth pattern as run_eval.py -
uses the session's Claude Code auth, no separate ANTHROPIC_API_KEY needed).
"""

import argparse
import json
import os
import re
import subprocess
import sys
from pathlib import Path

from scripts.utils import parse_skill_md

def _call_claude(prompt: str, model: str | None, timeout: int = 300) -> str:
    """Run `claude -p` with the prompt on stdin and return the text response.

    Prompt goes over stdin (not argv) because it embeds the full SKILL.md
    body and can easily exceed comfortable argv length.
    """
    cmd = ["claude", "-p", "--output-format", "text"]
    if model:
        cmd.extend(["--model", model])

    # Remove CLAUDECODE env var to allow nesting claude -p inside a
    # Claude Code session. The guard is for interactive terminal conflicts;
    # programmatic subprocess usage is safe. Same pattern as run_eval.py.
    env = {k: v for k, v in os.environ.items() if k != "CLAUDECODE"}

    result = subprocess.run(
        cmd,
        input=prompt,
        capture_output=True,
        text=True,
        env=env,
        timeout=timeout,
    )
    if result.returncode != 0:
        raise RuntimeError(
            f"claude -p exited {result.returncode}\nstderr: {result.stderr}"
        )
    return result.stdout

def improve_description(
    skill_name: str,
    skill_content: str,
    current_description: str,
    eval_results: dict,
    history: list[dict],

```

```

model: str,
test_results: dict | None = None,
log_dir: Path | None = None,
iteration: int | None = None,
) -> str:
    """Call Claude to improve the description based on eval results."""
    failed_triggers = [
        r for r in eval_results["results"]
        if r["should_trigger"] and not r["pass"]
    ]
    false_triggers = [
        r for r in eval_results["results"]
        if not r["should_trigger"] and not r["pass"]
    ]

    # Build scores summary
    train_score =
f"{eval_results['summary']['passed']}/{eval_results['summary']['total']}"
    if test_results:
        test_score =
f"{test_results['summary']['passed']}/{test_results['summary']['total']}"
        scores_summary = f"Train: {train_score}, Test: {test_score}"
    else:
        scores_summary = f"Train: {train_score}"

    prompt = f"""You are optimizing a skill description for a Claude Code skill called
"{skill_name}". A "skill" is sort of like a prompt, but with progressive disclosure --
there's a title and description that Claude sees when deciding whether to use the
skill, and then if it does use the skill, it reads the .md file which has lots more
details and potentially links to other resources in the skill folder like helper files
and scripts and additional documentation or examples.

The description appears in Claude's "available_skills" list. When a user sends a
query, Claude decides whether to invoke the skill based solely on the title and on
this description. Your goal is to write a description that triggers for relevant
queries, and doesn't trigger for irrelevant ones.

Here's the current description:
<current_description>
"{current_description}"
</current_description>

Current scores ({scores_summary}):
<scores_summary>
"""
    if failed_triggers:
        prompt += "FAILED TO TRIGGER (should have triggered but didn't):\n"
        for r in failed_triggers:
            prompt += f' - "{r["query"]}" (triggered {r["triggers"]}/{r["runs"]}
times)\n'
        prompt += "\n"

    if false_triggers:
        prompt += "FALSE TRIGGERS (triggered but shouldn't have):\n"
        for r in false_triggers:

```

```

        prompt += f' - "{r["query"]}" (triggered {r["triggers"]}/{r["runs"]}}
times)\n'
        prompt += "\n"

    if history:
        prompt += "PREVIOUS ATTEMPTS (do NOT repeat these - try something structurally
different):\n\n"
        for h in history:
            train_s = f"{h.get('train_passed', h.get('passed',
0))}/{h.get('train_total', h.get('total', 0))}"
            test_s = f"{h.get('test_passed', '?')}/{h.get('test_total', '?')}" if
h.get('test_passed') is not None else None
            score_str = f"train={train_s}" + (f", test={test_s}" if test_s else "")
            prompt += f"<attempt {score_str}>\n"
            prompt += f"Description: {h['description']}\n"
            if "results" in h:
                prompt += "Train results:\n"
                for r in h["results"]:
                    status = "PASS" if r["pass"] else "FAIL"
                    prompt += f' [{status}] "{r["query"][:80]}" (triggered
{r["triggers"]}/{r["runs"]})\n'
                if h.get("note"):
                    prompt += f'Note: {h["note"]}\n'
            prompt += "</attempt>\n\n"

    prompt += f"</scores_summary>"

```

```

Skill content (for context on what the skill does):
<skill_content>
{skill_content}
</skill_content>

```

Based on the failures, write a new and improved description that is more likely to trigger correctly. When I say "based on the failures", it's a bit of a tricky line to walk because we don't want to overfit to the specific cases you're seeing. So what I DON'T want you to do is produce an ever-expanding list of specific queries that this skill should or shouldn't trigger for. Instead, try to generalize from the failures to broader categories of user intent and situations where this skill would be useful or not useful. The reason for this is twofold:

1. Avoid overfitting
2. The list might get loooong and it's injected into ALL queries and there might be a lot of skills, so we don't want to blow too much space on any given description.

Concretely, your description should not be more than about 100-200 words, even if that comes at the cost of accuracy. There is a hard limit of 1024 characters - descriptions over that will be truncated, so stay comfortably under it.

Here are some tips that we've found to work well in writing these descriptions:

- The skill should be phrased in the imperative -- "Use this skill for" rather than "this skill does"
- The skill description should focus on the user's intent, what they are trying to achieve, vs. the implementation details of how the skill works.
- The description competes with other skills for Claude's attention - make it distinctive and immediately recognizable.

- If you're getting lots of failures after repeated attempts, change things up. Try different sentence structures or wordings.

I'd encourage you to be creative and mix up the style in different iterations since you'll have multiple opportunities to try different approaches and we'll just grab the highest-scoring one at the end.

Please respond with only the new description text in <new_description> tags, nothing else."""

```
text = _call_claude(prompt, model)

match = re.search(r"<new_description>(.*?)</new_description>", text, re.DOTALL)
description = match.group(1).strip().strip('\"') if match else
text.strip().strip('\"')

transcript: dict = {
    "iteration": iteration,
    "prompt": prompt,
    "response": text,
    "parsed_description": description,
    "char_count": len(description),
    "over_limit": len(description) > 1024,
}

# Safety net: the prompt already states the 1024-char hard limit, but if
# the model blew past it anyway, make one fresh single-turn call that
# quotes the too-long version and asks for a shorter rewrite. (The old
# SDK path did this as a true multi-turn; `claude -p` is one-shot, so we
# inline the prior output into the new prompt instead.)
if len(description) > 1024:
    shorten_prompt = (
        f"{prompt}\n\n"
        f"---\n\n"
        f"A previous attempt produced this description, which at "
        f"{len(description)} characters is over the 1024-character hard limit:\n\n"
        f'"{description}"\n\n'
        f"Rewrite it to be under 1024 characters while keeping the most "
        f"important trigger words and intent coverage. Respond with only "
        f"the new description in <new_description> tags."
    )
    shorten_text = _call_claude(shorten_prompt, model)
    match = re.search(r"<new_description>(.*?)</new_description>", shorten_text,
re.DOTALL)
    shortened = match.group(1).strip().strip('\"') if match else
shorten_text.strip().strip('\"')

    transcript["rewrite_prompt"] = shorten_prompt
    transcript["rewrite_response"] = shorten_text
    transcript["rewrite_description"] = shortened
    transcript["rewrite_char_count"] = len(shortened)
    description = shortened

transcript["final_description"] = description
```

```

    if log_dir:
        log_dir.mkdir(parents=True, exist_ok=True)
        log_file = log_dir / f"improve_iter_{iteration or 'unknown'}.json"
        log_file.write_text(json.dumps(transcript, indent=2))

    return description

def main():
    parser = argparse.ArgumentParser(description="Improve a skill description based on
eval results")
    parser.add_argument("--eval-results", required=True, help="Path to eval results
JSON (from run_eval.py)")
    parser.add_argument("--skill-path", required=True, help="Path to skill directory")
    parser.add_argument("--history", default=None, help="Path to history JSON (previous
attempts)")
    parser.add_argument("--model", required=True, help="Model for improvement")
    parser.add_argument("--verbose", action="store_true", help="Print thinking to
stderr")
    args = parser.parse_args()

    skill_path = Path(args.skill_path)
    if not (skill_path / "SKILL.md").exists():
        print(f"Error: No SKILL.md found at {skill_path}", file=sys.stderr)
        sys.exit(1)

    eval_results = json.loads(Path(args.eval_results).read_text())
    history = []
    if args.history:
        history = json.loads(Path(args.history).read_text())

    name, _, content = parse_skill_md(skill_path)
    current_description = eval_results["description"]

    if args.verbose:
        print(f"Current: {current_description}", file=sys.stderr)
        print(f"Score:
{eval_results['summary']['passed']}/{eval_results['summary']['total']}",
file=sys.stderr)

    new_description = improve_description(
        skill_name=name,
        skill_content=content,
        current_description=current_description,
        eval_results=eval_results,
        history=history,
        model=args.model,
    )

    if args.verbose:
        print(f"Improved: {new_description}", file=sys.stderr)

    # Output as JSON with both the new description and updated history
    output = {
        "description": new_description,

```

```
        "history": history + [{
            "description": current_description,
            "passed": eval_results["summary"]["passed"],
            "failed": eval_results["summary"]["failed"],
            "total": eval_results["summary"]["total"],
            "results": eval_results["results"],
        }],
    }
    print(json.dumps(output, indent=2))

if __name__ == "__main__":
    main()
```